

3.2 Core Concepts of OOP

Object-Oriented Programming (OOP) has several core concepts that make programs organized, reusable, and easier to manage. These concepts include classes and objects, encapsulation, inheritance, and polymorphism.

Classes and Objects - Representing Real-World Entities in Code

Class: A class is a blueprint or template that defines the properties (attributes) and actions (methods) of an entity.

Object: An object is an instance of a class that holds actual values for its attributes.

Key Point: A class is like a blueprint; an object is the real thing created from that blueprint. Classes and objects allow programmers to model real-world entities in code. Using classes and objects helps organize data and related functionality together, making programs more structured.

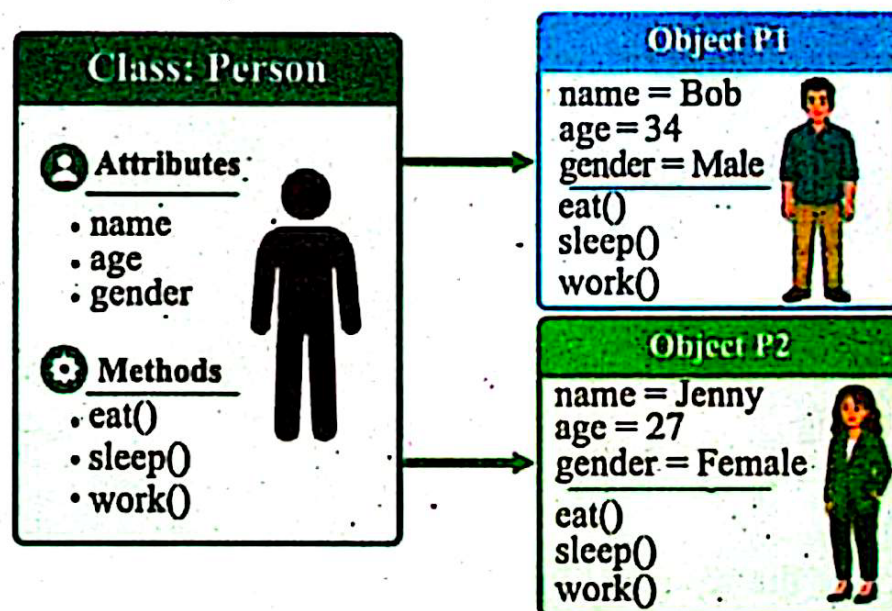


Figure 3.1: Diagram Showing Class and Object relationship

Example Code:

```
class Student:
    def __init__(self, name):
        self.name = name # stores the student's name

    def display(self):
        print("Student Name:", self.name)
```

```
# Creating objects
object1 = Student("Ali")
object2 = Student("Ahmed")

# Using objects
object1.display()
object2.display()
```

'class Student' → defines a blueprint for students.

'__init__' → sets the name when an object is created.

'self' is the reference to the current object, passed automatically to instance methods.

self.name → stores the student's name

display() → prints the student's name

object1 and object2 → two different student objects

Each object has its own data (Ali, Ahmed)

Difference Between Class and Object

Class	Object
A class is a blueprint for declaring and creating objects.	An object is a class instance that allows programmers to use variables and methods from inside the class.
Memory is not allocated to classes. Classes have no physical existence.	When objects are created, memory is allocated to them in the heap memory.
You can declare a class only once.	A class can be used to create many objects.
Class is a logical entity.	An object is a physical entity.
We cannot manipulate class as it is not available in memory.	Objects can be manipulated.
Class is created using the class keyword like class Dog	Objects are created like dog = Dog() .
Example: Mobile is a class.	If Mobile is the class then iphone , blackberry , samsung are its objects which have different properties and behaviours.

Class	Object
Classes can have attributes and methods defined.	Objects can have specific values assigned to their attributes.
Classes serve as blueprints for creating objects.	Objects represent specific instances created from a class.

Short Questions

1. What is a class?
2. What is an object?
3. Differentiate between class and object.
4. What is the purpose of `__init__` method?
5. What is the purpose of `self` in Python?
6. Give one real-life example of class and object.

Long Questions

1. Explain the concept of classes and objects in OOP with a suitable example.
2. Write a Python program to create a class `Student` with `name` and `display` method. Create two objects and display their names.